UNIT-3

FIRST ORDER LOGIC

3.4 Syntax and semantics of First Order Logic

3.4.1 Models for first-order logic

- Models in logic describe possible worlds by linking vocabulary (symbols) to objects and their relations.
- Domain = set of objects in the model (must be nonempty).
- A model = domain , relations , functions, which lets us decide truth of first-order logic sentences.

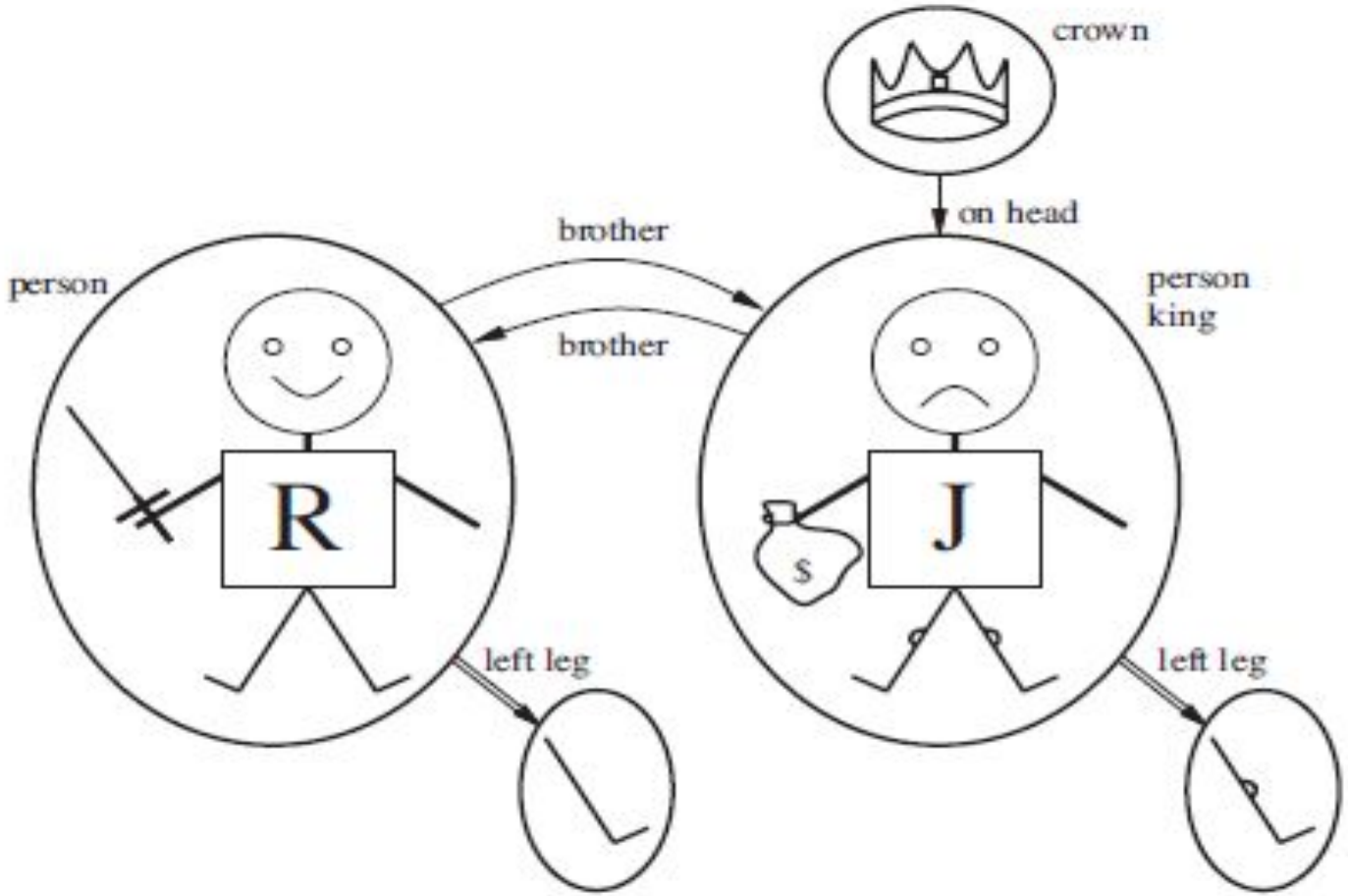


Figure 8.2 A model containing five objects, two binary relations, three unary relations (indicated by labels on the objects), and one unary function, left-leg.

- Example model (Fig 8.2) has 5 objects: Richard, John, Richard's left leg, John's left leg, crown.

Binary relations:

- Brother = {⟨Richard, John⟩, ⟨John, Richard⟩}
- On head = {⟨crown, John⟩}

Unary relations (properties):

- Person: Richard, John
- King: John
- Crown: crown

Unary function:

- Left leg: Richard → Richard's left leg; John → John's left leg

- Example model (Fig 8.2) has 5 objects: Richard, John, Richard's left leg, John's left leg, crown.

Binary relations:

- Brother = {⟨Richard, John⟩, ⟨John, Richard⟩}
- On head = {⟨crown, John⟩}

Unary relations (properties):

- Person: Richard, John
- King: John
- Crown: crown

Unary function:

- Left leg: Richard → Richard's left leg; John → John's left leg

- Richard and John are brothers. Formally speaking, a relation TUPLE is just the set of tuples of objects that are related. (A tuple is a collection of objects arranged in a fixed order and is written with angle brackets surrounding the objects.) Thus, the brotherhood relation in this model is the set
  - { Richard the Lionheart, King John, King John, Richard the Lionheart }
- For example, each person has one left leg, so the model has a unary "left leg" function that includes the following mappings:
  - {Richard the Lionheart}⟶ Richard's left leg
  - {King John} ⟶ John's left leg .
- Strictly speaking, models in first-order logic require total functions, that is, there must be a value for every input tuple.

3.4.2 Symbols and interpretations

a)What a model is in FOL:

- A model consists of:
  - Objects → the elements of the domain.
  - Interpretation → mapping of:
    - ✔ Constant symbols → to specific objects.
    - ✔ Predicate symbols → to relations among objects.
    - ✔ Function symbols → to functions on objects.

b) Interpretations

- Different models are possible because the mapping is flexible.
- Example:
  - Richard → Richard the Lionheart.
  - John → King John.
  - Brother → brotherhood relation.
  - OnHead → relation between crown and King John.
  - Person, King, Crown → sets of objects.
  - LeftLeg → a mapping from a person to their left leg.
- Other mappings are possible (e.g., Richard → crown, John → a leg).

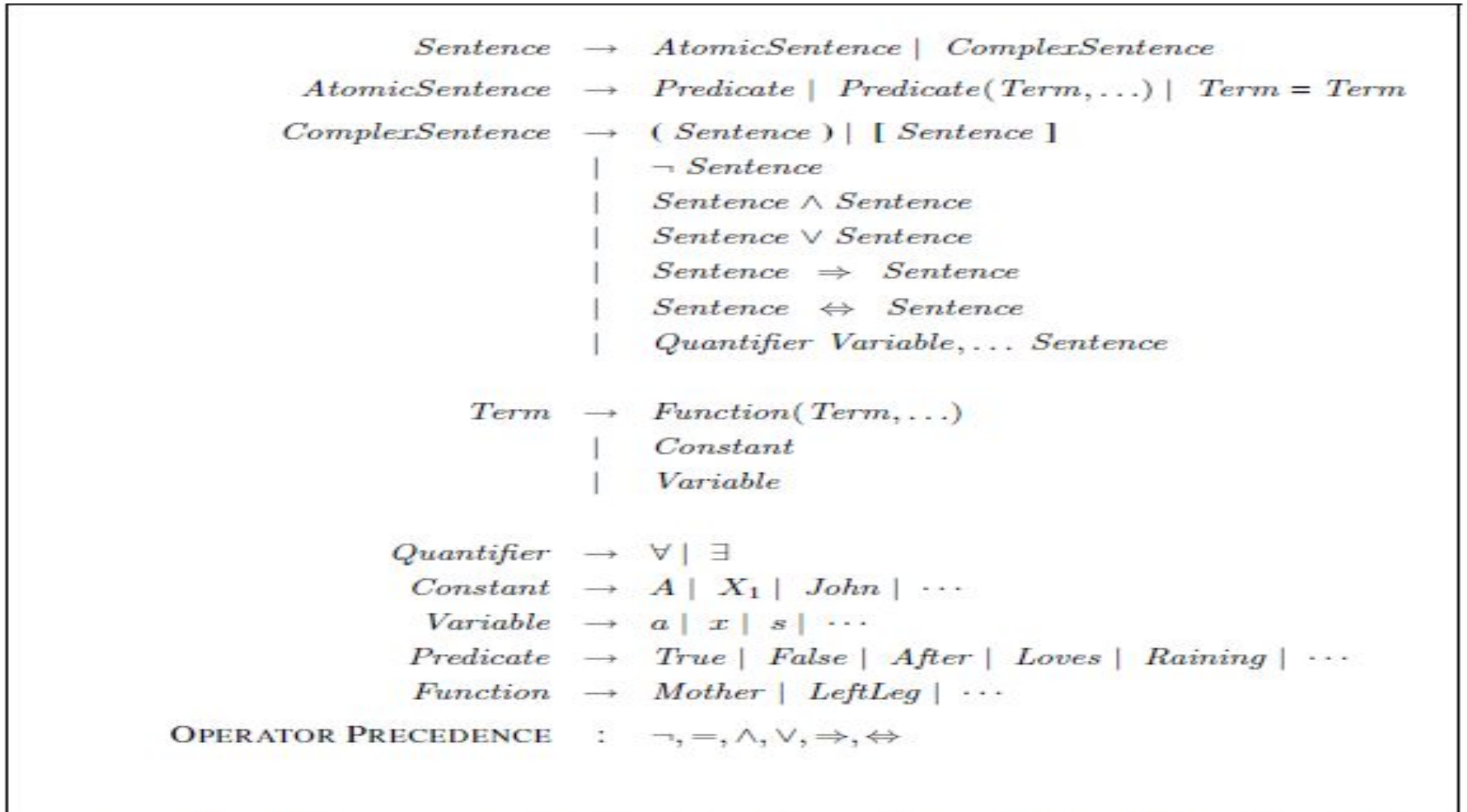
- Objects may:
  - Have no names (unnamed objects exist in the domain).
  - Have multiple names (two constants can refer to the same object).

c) Variety of Models

- Models vary by:
  - Number of objects (from 1 to infinity).
  - How constants are mapped to objects.
- Example: with two constants but only one object, both constants must refer to the same object.
- Even with more objects, constants can still point to the same object.

d) Why model checking is hard

- In propositional logic, models are finite and enumerable, so entailment can be checked directly.
- In first-order logic:
  - The number of possible models is unbounded.
  - Even if restricted to a small number of objects, combinations grow very large.
  - Example: with ≤ 6 objects, there are 137,506,194,466 possible models (from Fig. 8.4).
- Hence, entailment cannot be decided by brute-force enumeration.
- A model in FOL = domain (objects) + interpretation (mapping constants → objects, predicates → relations, functions → functions).



$$
\begin{aligned}
\textit{Sentence} &\rightarrow \textit{AtomicSentence} \mid \textit{ComplexSentence} \\
\textit{AtomicSentence} &\rightarrow \textit{Predicate} \mid \textit{Predicate}(\textit{Term}, \ldots) \mid \textit{Term} = \textit{Term} \\
\textit{ComplexSentence} &\rightarrow (\ \textit{Sentence}\ ) \mid [\ \textit{Sentence}\ ] \\
&\mid \neg\ \textit{Sentence} \\
&\mid \textit{Sentence} \wedge \textit{Sentence} \\
&\mid \textit{Sentence} \vee \textit{Sentence} \\
&\mid \textit{Sentence} \Rightarrow \textit{Sentence} \\
&\mid \textit{Sentence} \Leftrightarrow \textit{Sentence} \\
&\mid \textit{Quantifier}\ \textit{Variable}, \ldots\ \textit{Sentence} \\
\textit{Term} &\rightarrow \textit{Function}(\textit{Term}, \ldots) \\
&\mid \textit{Constant} \\
&\mid \textit{Variable} \\
\textit{Quantifier} &\rightarrow \forall \mid \exists \\
\textit{Constant} &\rightarrow A \mid X_1 \mid \textit{John} \mid \cdots \\
\textit{Variable} &\rightarrow a \mid x \mid s \mid \cdots \\
\textit{Predicate} &\rightarrow \textit{True} \mid \textit{False} \mid \textit{After} \mid \textit{Loves} \mid \textit{Raining} \mid \cdots \\
\textit{Function} &\rightarrow \textit{Mother} \mid \textit{LeftLeg} \mid \cdots \\
\textsc{Operator Precedence} &: \neg, =, \wedge, \vee, \Rightarrow, \Leftrightarrow
\end{aligned}
$$

Figure 8.3 The syntax of first-order logic with equality, specified in Backus–Naur form (see page 1060 if you are not familiar with this notation). Operator precedences are specified, from highest to lowest. The precedence of quantifiers is such that a quantifier holds over everything to the right of it.

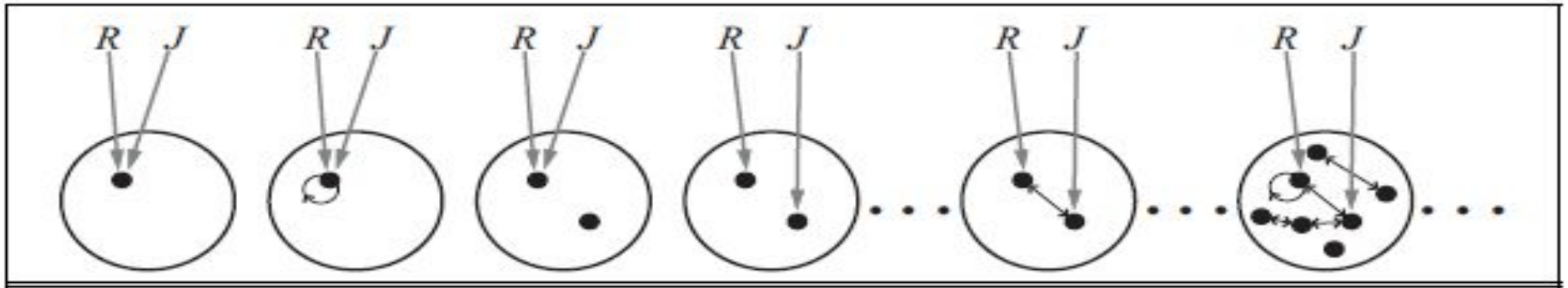


Figure 8.4 Some members of the set of all models for a language with two constant symbols, R and J, and one binary relation symbol. The interpretation of each constant symbol is shown by a gray arrow. Within each model, the related objects are connected by arrows.

3.4.3 Terms

Term: A term is any logical expression that refers to an object in the domain.

Examples:

- Constant symbols (e.g., John, Richard).Function-based expressions (e.g., LeftLeg(John)).
- Function-based expressions (e.g., LeftLeg(John)).

- Function Symbols are used because
  - Not every object has its own name (constant).
  - Function symbols let us describe objects without introducing new constants.
- **Example:** instead of naming "John's left leg," we write LeftLeg(John).

Complex Term

- A complex term = function symbol and arguments (which are themselves terms).

  General form: f(t1, …, tn).

- Example: LeftLeg(John) → the left leg of John.

- Function symbols are not subroutines (unlike in programming).
  - They don't "compute" anything.
  - They just name an object in the domain, as defined by the model's interpretation.
  - Example: LeftLeg is a mapping already fixed by the model.

Semantics of terms:

- If f refers to a function F in the model:
  - $t_1, \ldots, t_n \rightarrow$ refer to objects $d_1, \ldots, d_n$ in the domain.
  - Then $f(t_1, \ldots, t_n)$ refers to the object $F(d_1, \ldots, d_n)$.

- Example:
  - If John → King John.
  - If LeftLeg → mapping from people to their left legs.
  - Then LeftLeg(John) → King John's left leg.
- A term = name for an object.
- Constants are simple terms; function symbols create complex terms. Interpretation tells us exactly which object each term refers to.

3.4.4 Atomic sentences

- Atomic Sentence (or atom for short) : predicate symbol optionally followed by a parenthesized list of terms, such as

  Brother (Richard , John).

- This states, under the intended interpretation given earlier, that Richard the Lionheart is the brother of King John.6 Atomic sentences can have complex terms as arguments. Thus,

  Married(Father (Richard),Mother (John))

- states that Richard the Lionheart's father is married to King John's mother (again, under a suitable interpretation).

3.4.5 Complex sentences

- We can use logical connectives to construct more complex sentences, with the same syntax and semantics as in propositional calculus.
  - ¬Brother (LeftLeg(Richard), John)
  - Brother (Richard , John) $\wedge$ Brother (John,Richard)
  - King(Richard) $\vee$ King(John)
  - ¬King(Richard) $\Rightarrow$ King(John) .

3.4.6 Quantifiers

First-order logic contains two standard quantifiers, called universal and existential.

Universal quantification ($\forall$)

- In first-order logic, universal quantification allows us to express general rules like "All kings are persons," written as $\forall x$ King(x) $\Rightarrow$ Person(x).
- This means that for every object x, if x is a king, then x is a person. A variable (like x) can stand alone as a term or appear inside a function (e.g., LeftLeg(x)). A ground term is a term with no variables.
- The sentence $\forall x$ P is true in a model if P holds under all possible assignments of x to objects in the domain.
- For example, with objects {Richard, John, Richard's left leg, John's left leg, crown}, the rule expands into separate implications for each object.

- Only King John satisfies the premise “is a king,” so the rule asserts that he is a person.
- For non-kings like legs or the crown, the premise is false, making the implication true without saying anything about them.
- A common mistake is to write $\forall x$ King(x) $\wedge$ Person(x), which wrongly asserts that every object is both a king and a person.
- The correct use of implication ($\Rightarrow$) ensures the rule applies only to actual kings.
- Implication ($\Rightarrow$) is true whenever something is not a king, so it only “checks” kings.
- Conjunction ($\wedge$) wrongly forces everything to be both a king and a person.

King(x)	Person(x)	King(x) ⇒ Person(x)	King(x) ∧ Person(x)
T	T	T ✅ (correct: king is a person)	T ✅
T	F	F ❌ (wrong: king but not a person)	F ❌
F	T	T ✅ (since not a king, rule holds vacuously)	F ❌ (forces non-kings to be kings too)
F	F	T ✅ (since not a king, rule holds vacuously)	F ❌

Existential quantification (∃)

- The existential quantifier (∃) is used to state that something is true for at least one object in the domain.

- For example, $\exists x$ Crown(x) $\wedge$ OnHead(x, John) means "There exists an x such that x is a crown and x is on John's head."
- This sentence is true in a model if at least one object satisfies both conditions. In the example, the crown itself satisfies them, so the sentence is true. If John wore two crowns, the sentence would still be true, since it only requires at least one.
- Just as implication ($\Rightarrow$) works naturally with $\forall$, conjunction ($\wedge$) works naturally with $\exists$.
- If we instead wrote $\exists x$ Crown(x) $\Rightarrow$ OnHead(x, John), the meaning becomes too weak.
- This is because an implication is true whenever the premise is false, so the sentence would hold as soon as we find any object that is not a crown—something almost always available in the domain. Thus, the correct way to express existential statements is with $\wedge$, not $\Rightarrow$.

- $\forall \rightarrow$ general rules, usually with $\Rightarrow$.
- $\exists \rightarrow$ specific existence, usually with $\wedge$.
- **Nested quantifiers** let us combine multiple quantifiers in a single sentence.
- If the quantifiers are the same type, we can combine them
  - Example: $\forall x \ \forall y$ Brother(x, y) $\Rightarrow$ Sibling(x, y) ("Brothers are siblings").
  - Equivalent shorthand: $\forall x, y$ Sibling(x, y) $\Leftrightarrow$ Sibling(y, x) (siblinghood is symmetric).
- With mixed quantifiers, order matters:
  - $\forall x \ \exists y$ Loves(x, y) $\rightarrow$ everyone loves someone.
  - $\exists y \ \forall x$ Loves(x, y) $\rightarrow$ there is someone loved by everyone.
- Parentheses clarify scope: $\forall x \ (\exists y$ Loves(x, y)) vs. $\exists y \ (\forall x$ Loves(x, y)).

- Variables are bound by their closest quantifier. For example,
  - $\forall x$ (Crown(x) $\vee$ ($\exists x$ Brother(Richard, x))) really means
  - $\forall x$ (Crown(x) $\vee$ ($\exists z$ Brother(Richard, z))).
- To avoid confusion, always use different variable names when nesting quantifiers.

2. Existential Quantifier ($\exists x$ Crown(x) $\wedge$ OnHead(x, John))

Crown(x)	OnHead(x, John)	Crown(x) $\wedge$ OnHead(x, John)	Notes
T	T	✅ True	A crown is on John's head.
T	F	❌ False	Crown not on head.
F	T	❌ False	On head but not a crown.
F	F	❌ False	Neither.

👉 With $\exists$, **at least one true case** is enough.

Connections between $\forall$ and $\exists$

- The universal ($\forall$) and existential ($\exists$) quantifiers are connected through negation, much like $\wedge$ and $\vee$ in propositional logic. For example:
- $\forall x \neg \text{Likes}(x, \text{Parsnips}) \equiv \neg \exists x \text{ Likes}(x, \text{Parsnips})$ ("everyone dislikes parsnips" ≡ "no one likes parsnips").
- $\forall x \text{ Likes}(x, \text{IceCream}) \equiv \neg \exists x \neg \text{Likes}(x, \text{IceCream})$ ("everyone likes ice cream" ≡ "there is no one who doesn't").
- These follow De Morgan's rules for quantifiers:
  - $\forall x \neg P \equiv \neg \exists x P$ $\quad\quad$ $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$
  - $\neg \forall x P \equiv \exists x \neg P$ $\quad\quad$ $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$
  - $\forall x P \equiv \neg \exists x \neg P$ $\quad\quad$ $P \wedge Q \equiv \neg(\neg P \vee \neg Q)$
  - $\exists x P \equiv \neg \forall x \neg P$ $\quad\quad$ $P \vee Q \equiv \neg(\neg P \wedge \neg Q)$

- Thus, either quantifier could be defined in terms of the other, but both are kept for clarity and readability.

3.4.7 Equality

- In first-order logic, we can form atomic sentences using the equality symbol (=), which asserts that two terms refer to the same object.
- For example, Father(John) = Henry states that John's father is Henry.
- Equality can also be negated to show that two terms are different. For instance, to say Richard has at least two brothers, we write: $\exists$ x, y Brother(x, Richard) $\wedge$ Brother(y, Richard) $\wedge$ $\neg$(x = y).
- Without the inequality, the statement could be satisfied even if Richard had only one brother, since both variables might refer to the same person. The shorthand $x \neq y$ is often used instead of $\neg(x = y)$.

3.4.8 An alternative semantics?

- Simply writing Brother(John, Richard) $\wedge$ Brother(Geoffrey, Richard) does not fully capture the fact that Richard's brothers are exactly John and Geoffrey.
- We must also assert that John $\neq$ Geoffrey and that no one else is Richard's brother:
- Brother(John, Richard) $\wedge$ Brother(Geoffrey, Richard) $\wedge$ John $\neq$ Geoffrey $\wedge$ $\forall x$ (Brother(x, Richard) $\Rightarrow$ (x = John $\vee$ x = Geoffrey)).
- This is more cumbersome than natural language. To simplify, database semantics is often used, which assumes:
  - Unique-names assumption $\rightarrow$ each constant refers to a distinct object.
  - Closed-world assumption $\rightarrow$ any fact not known to be true is false.
  - Domain closure $\rightarrow$ the domain has only the objects named by constants.

- Under these rules, the simpler sentence suffices to mean that Richard's only brothers are John and Geoffrey.
- Database semantics reduces the number of possible models and works well when we know all the objects and facts, but in general reasoning, standard first-order semantics is preferred despite being more verbose.

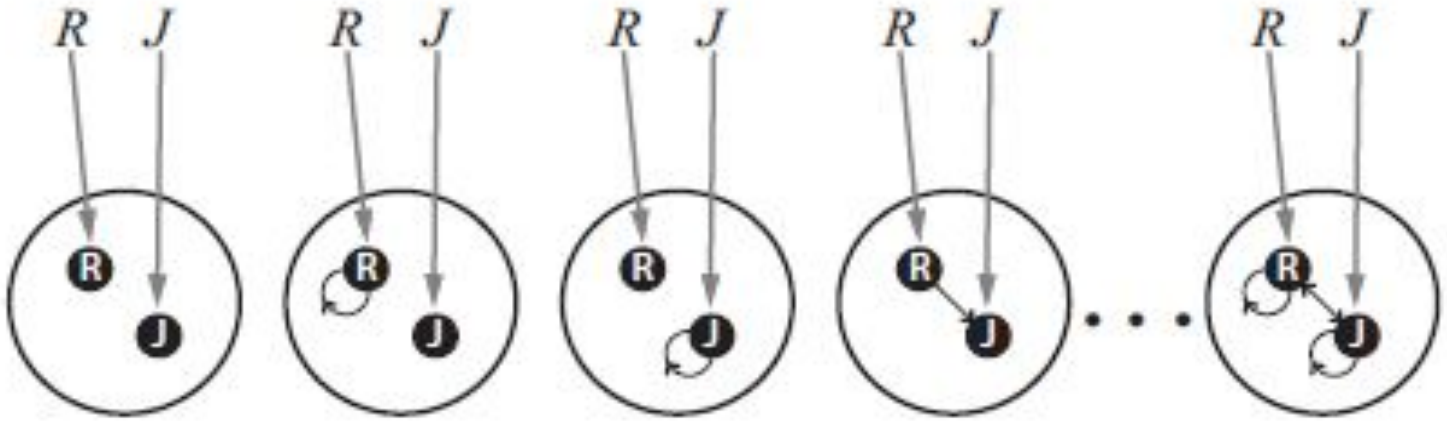


Figure 8.5 Some members of the set of all models for a language with two constant symbols, R and J, and one binary relation symbol, under database semantics. The interpretation of the constant symbols is fixed, and there is a distinct object for each constant symbol.